INTERNAL OPERATIONS GUIDE

SJC

Singapore Journal of Cardiology
Hostinger knowledge-transfer guide

A practical guide to locating the live application, understanding its technology, making changes, and maintaining data and background jobs on Hostinger.

Prepared: 24 October 2026 **Hosting layout last verified:** 19 October 2026

The three locations to remember

Domain root

```
/home/u709299470/domains/sjcjournal.com
```

Application code

```
/home/u709299470/domains/sjcjournal.com/journal_app
```

Public website root

```
/home/u709299470/domains/sjcjournal.com/public_html
```

Website: https://sjcjournal.com | **Hosting control panel:** hpanel.hostinger.com

Guide contents

02 Hostinger folder map

03 Application folders and responsibilities

04 Where to make common changes

05 Database, uploads and configuration

06 Production and build technology

07 Cron jobs, email and queues

08 Deployment, backups and recovery

09 Maintenance and troubleshooting

10 Handover checklist and evidence

02 / HOSTINGER HANDOVER

Where the live code lives

In hPanel, open **Websites > sjcjournal.com > Files > File Manager > Access files of sjcjournal.com**. The domain-level view shows the private application and public web folders as siblings.

```
/home/u709299470/domains/sjcjournal.com
```

```
domain root/
+-- journal_app/                          PRIVATE APPLICATION
|   +-- app/                             PHP application code
|   +-- bootstrap/                       Laravel startup and cache
|   +-- config/                          Configuration definitions
|   +-- content/journal-pages/           Markdown content resources
|   +-- database/                        Migrations and database resources
|   +-- resources/                       Blade, CSS and JavaScript source
|   +-- routes/                          Web, portal, API and console routes
|   +-- scripts/                         Guarded mail-worker entry point
|   +-- storage/                         Files, runtime data and logs
|   +-- vendor/                          Installed PHP dependencies
|   +-- .env                             Private runtime settings
|   +-- artisan                          Laravel command entry point
|   +-- composer.json / composer.lock    PHP dependency definitions
|   +-- package.json / package-lock.json Frontend dependency definitions
|   +-- public/                          Bundled public source copy
+-- public_html/                         ACTIVE PUBLIC WEB ROOT
|   +-- index.php                        Loads journal_app
|   +-- .htaccess                        Web rules and asset MIME settings
|   +-- build/                           Compiled assets and manifest
|   +-- assets/                          Logos, icons and placeholders
|   +-- storage/                         Public upload entry/mapping
|   +-- author-resources/                Public forms/resources
|   +-- images/ / errors/                Public supporting files
+-- journal-backup-*/                    Private release backups
+-- .<release>.stage / .<release>.done   Release staging and result markers
```

ACTIVE WEB ROOT

public_html is the verified document root. journal_app/public is a bundled source directory; changing that copy alone may not update the publicly served website. Preserve the production index.php and Laravel public-path binding.

Historical folders including **journal-live**, **journal-release** and **public_html-before-journal** were also observed. Review their contents and retention before cleanup; the recorded active application root is journal_app.

03 / HOSTINGER HANDOVER

Application folder responsibilities

All paths on this page are relative to **journal_app/**. This is where server-side source and editable interface templates are stored.

Folder / file	Purpose and maintenance guidance
`app/Http/Controllers/`	Handles public pages, account actions, role portals, administration and API requests.
`app/Http/Requests/` and `app/Http/Middleware/`	Validates input; checks account state, roles, upload rules and request security.
`app/Models/`	Database entities and relationships: articles, users, authors, reviews, workflow files, settings and queue-related records.
`app/Services/`	Business rules: manuscript transitions, publication, uploads, search, SEO and notification coordination.
`app/Policies/` and `app/Providers/`	Record-level authorization, role gates, service bindings and event registration.
`app/Events/`, `app/Listeners/`, `app/Notifications/`	Workflow events, in-app notices and queued email behavior.
`resources/views/`	Blade templates for public pages, shared components, layouts, emails and role workspaces.
`resources/css/` and `resources/js/`	CSS and JavaScript source. Build requirements depend on which file changes; see page 4.
`routes/`	public.php for reader pages; auth.php for identity; portal.php for role workspaces; api.php for API routes; console.php for schedules.
`config/`	Configuration definitions, including publication.php, workflow.php, journal_pages.php, filesystems.php and security.php.
`content/journal-pages/`	Markdown guidance resources. Published page state/content is also stored in database settings.
`database/migrations/`	Schema history. This folder is not proof of the active SQLite filename; use the private DB_DATABASE setting.
`bootstrap/`, `vendor/`, `artisan`	Startup and runtime support. vendor contains Composer-installed dependencies, not the place to customize journal behavior.

PATH CONVENTION

In this guide, an application-relative path such as resources/views/public/home.blade.php means that path inside /home/u709299470/domains/sjcjournal.com/journal_app/. Public paths are explicitly labelled public_html/.

04 / HOSTINGER HANDOVER

Where to make common changes

Use the journal administration screens for data and publication changes. Use application files for templates, presentation and business behavior.

Change needed	Primary location	What it controls
Homepage	`resources/views/public/home.blade.php`	Page sections, hero markup and article/editor presentation.
Header and shared page layout	`resources/views/layouts/public.blade.php`	Journal identity, navigation, asset loading and common page structure.
Footer	`resources/views/public/partials/footer.blade.php`	Shared footer links, contact area and journal updates.
Margins and responsive spacing	`resources/css/public-layout.css`	4 cm desktop gutters above 1200 px; 24 px tablet and 16 px phone gutters.
Article and editor cards	`resources/views/components/public/`	article-card.blade.php and editor-card.blade.php define repeated card markup.
Archives and article reader	`resources/views/public/articles/`	archive.blade.php, issue.blade.php and show.blade.php; Public controllers supply data.
About, editorial and guidance views	`resources/views/public/pages/`	Page templates. Editorial records, article metadata and published guidance also use database records.
Navigation / client interactions	`resources/js/app.js`	Mobile menus, search, carousel and browser enhancements.
PDF reader behavior	`resources/js/article-reader.js`	PDF.js loading, rendered pages and reader controls.
Workflows / account access	`app/Services/, app/Policies/ and app/Http/`	Editorial transitions, permissions, account verification and approval rules.

Two different asset-update paths

• **Inline public styles:** public-layout.css and journal-pages.css are read by the shared Blade layout. Deploy the changed source file and clear relevant Laravel/hosting caches.

• **Vite assets:** changes to resources/css/app.css or resources/js/ require a build. Deploy the matching output and manifest together into public_html/build/.

Published guidance: use the admin draft/publish workflow. JournalPages reads journal_page.* and legacy content settings; changing a Markdown file alone does not publish the page.

05 / HOSTINGER HANDOVER

Database, uploads and settings

The production database is SQLite. It stores accounts, articles, editorial records, submissions, reviews, workflow state, settings, notifications, queued jobs and failed jobs. Uploaded files are stored separately and must be backed up with the database.

LOCATE THE ACTIVE DATABASE
The active filename is the private DB_DATABASE value used by journal_app/.env and the loaded database configuration. Its exact path was not freshly confirmed for this guide. Resolve that value before backup or restore; do not assume a filename from a Laravel default.

Area	Location / path convention	Access and meaning
Runtime configuration	`journal_app/.env`	Private settings and secrets. Keep outside the public web root.
Private filesystem root	`journal_app/storage/app/private/`	Default local disk; controlled manuscripts, review files and workflow documents.
New manuscript uploads	`articles/manuscripts/ on the local disk`	Physical path is under the private disk root. Published PDFs are located through articles.pdf_path.
Workflow / review files	`workflow/<article-id>/ and reviews/ on the local disk`	Keep attachment rows, relative paths and actual files together.
Public upload entry	`public_html/storage/`	Recorded serving location for the configured public disk. Preserve the existing mapping or binding.
Public image paths	`authors/avatars/, editors/avatars/, reviewers/avatars/, users/avatars/, articles/images/`	Relative to the configured public disk, not necessarily a literal directory copied under every root.
Media library	`media/YYYY/MM/ on the recorded disk`	The media row records disk, path, visibility and checksum. Resolve both disk and path.
Logs / runtime files	`journal_app/storage/logs/ and storage/framework/`	Application logs, compiled views, cache/session areas and worker locks/holds.

Configuration ownership

config/*.php contains configuration definitions; .env supplies environment-specific values; database settings contain editable journal content and options. Important key names include APP_ENV, APP_URL, APP_DEBUG, DB_CONNECTION, DB_DATABASE, CACHE_STORE, SESSION_DRIVER, QUEUE_CONNECTION and MAIL_*. Keep their sensitive values in protected configuration.

The standard source maps the public disk to storage/app/public. Production evidence confirms its resolved root matches public_html/storage. Verify the actual mapping before altering it.

06 / HOSTINGER HANDOVER

Technology stack and request flow

Versions below identify the reviewed lockfile package set. Installed server versions were not freshly inspected on 24 October.

Layer	Technology / version	Role
Web hosting / runtime	Hostinger shared hosting; PHP 8.4	HTTPS, PHP execution and scheduled cron. Composer requires PHP 8.4.1 or newer within 8.x.
Application	Laravel 13.34.0	Routing, validation, ORM, server-rendered pages, jobs and notifications.
User interface	Blade, Tailwind CSS 4.3.3, custom CSS, vanilla JavaScript	Public journal and role portals.
Production database	SQLite with pdo_sqlite	Private, file-based relational data store.
Authentication / access	Laravel sessions; Sanctum 4.3.3; custom roles and policies	Login, verification, token access, ownership and role restrictions.
Queues / scheduling	Laravel database queue and Hostinger cron	Email processing, publication checks and reminders.
Filesystem	Laravel Filesystem / Flysystem local disks	Private documents and publicly served images.
PDF viewing / generation	PDF.js 6.3.289; Dompdf 3.1.6	Browser reader and generated acceptance letters.
Mail	Hostinger SMTP over SMTPS	Authenticated notification delivery.
Frontend build tools	Node.js 22.13+ in 22.x; Vite 8.2.2; Laravel Vite plugin 3.2.0	Produces browser assets. A Node web-server process is not required for normal PHP requests.
Dependency tools	Composer 2 and npm lockfiles	Repeatable PHP and frontend dependency installation.
Optional integrations	GA4 with consent; Google identity; S3 adapter	Capabilities depend on private configuration. DOI handling uses the manual provider.
Domain / DNS	GoDaddy DNS in the recorded setup	Domain routing and mail DNS records; confirm current ownership before DNS work.

How a request reaches the application

```
Browser -> Hostinger HTTPS -> public_html/index.php
-> journal_app/bootstrap/app.php -> routes and middleware
-> controller / service -> SQLite and files -> Blade response
```

Redis and PostgreSQL belong to the development/reference environment. The recorded Hostinger topology uses SQLite and runtime-configured cache/session drivers.

07 / HOSTINGER HANDOVER

Cron jobs, email and queues

Two regular cron jobs were verified in Hostinger. Both run every minute using the PHP 8.4 executable. In hPanel, inspect **Advanced > Cron Jobs**. Enter each command as one command; wrapping below is visual.

Scheduler - expression: * * * * *

```
/opt/alt/php84/usr/bin/php /home/u709299470/domains/sjcjournal.com/journal_app/artisan
schedule:run
```

Guarded queue worker - expression: * * * * *

```
/opt/alt/php84/usr/bin/php
/home/u709299470/domains/sjcjournal.com/journal_app/scripts/hostinger-mail-worker.php
```

Schedule / limit	Current source behavior
Every minute	Checks scheduled publications through articles:publish-scheduled --sync.
Hourly	workflow:remind handles editorial workflow reminders.
Daily	sanctum:prune-expired removes expired tokens according to the command settings.
Mail worker	Processes mail,default; stops when empty, after 45 seconds or after 20 jobs.
Worker bounds	Timeout 30 seconds; sleep/rest 1 second; transient backoff 60 seconds; tries 3.
Concurrency / hold	storage/framework/mail-worker.lock prevents overlap; mail-paused deliberately pauses delivery.

Mail configuration and behavior

The recorded sender is **notifications@sjcjournal.com**, using smtp.hostinger.com on port 465 with SMTPS. Office routing uses publication settings and the configured office recipients. Credentials remain in private runtime configuration. A queued job being accepted does not establish inbox delivery.

PRESERVE HISTORICAL QUEUES
Older queues include mail-before-smtp-20261008, mail-before-repair-20261014 and maintenance-before-repair-20261014. The regular worker excludes held queues. Failed jobs are retained for review. Inspect individual jobs and recipients before any retry; do not bulk replay or prune them during routine maintenance.

Registration, email verification and Super Admin approval are separate controls. A verified email alone does not approve access; staff also need verified email before receiving new assignments.

08 / HOSTINGER HANDOVER

Deployments, backups and recovery

Use a controlled, incremental release appropriate to this custom Hostinger layout. Preserve the private database, environment, uploads and existing public-path binding throughout the update.

1. Identify the change. Map changed templates, PHP files, inline CSS or built assets to the correct application/public destination.

2. Capture a rollback set. Back up replaced source files, protected configuration, a consistent SQLite snapshot and all upload locations. Store backups outside public_html.

3. Verify the baseline. Check current file hashes and production paths before replacement. Stop for review if live source differs from the reviewed baseline.

4. Rehearse away from production. Test application boot, affected pages and rollback against isolated data and storage. Never run database-reset tests against the journal database.

5. Apply the update. Use atomic replacement where possible. Coordinate a short maintenance/mail hold if required, retaining the original hold state.

6. Refresh the runtime. Clear relevant view/route/config caches and restart workers. New PHP classes require a regenerated optimized Composer class map.

7. Verify the live result. Check real page copy, navigation, archives, images, PDF download and PDF canvas rendering. Compare retained records and uploads.

8. Close the release. Restore the regular worker and pre-existing service state. Record the result and retained backup location; keep rollback material until review is complete.

Recorded release-backup locations

```
Domain root / journal-backup-fluid-layout-release-20261019/
Domain root / journal-backup-desktop-spacing-release-20261019/
```

These release backups include **database-before.sqlite**, **records-before.json** and replaced source files beneath app/ and/or public/. The 19 October installer used a consistent SQLite snapshot and validated it with quick_check and row counts.

> **RESTORE THE RIGHT MATERIAL**
> database-before.sqlite is a release backup, not the documented active database filename. For a code rollback, restore the replaced files first. Restore a database only when data recovery is needed: isolate writes/workers, validate a copy, resolve DB_DATABASE, restore matching uploads and verify records before reopening access.

hPanel backup availability and retention should be checked in the account. Keep a protected off-host copy of database, uploads and configuration; a same-server release folder alone is not an off-host backup.

09 / HOSTINGER HANDOVER

Maintenance and troubleshooting

When authorized SSH/terminal access is available, start in the private application directory. Use the explicit PHP 8.4 executable instead of relying on a default PHP version.

```
cd /home/u709299470/domains/sjcjournal.com/journal_app
/opt/alt/php84/usr/bin/php artisan about
/opt/alt/php84/usr/bin/php artisan migrate:status
/opt/alt/php84/usr/bin/php artisan schedule:list
/opt/alt/php84/usr/bin/php artisan queue:failed
```

These are inspection commands. Runtime-changing commands, queue retries and schema operations belong to a reviewed maintenance action. Do not run migrate:fresh, migrate:refresh, db:wipe, seeders or tests against the live journal database.

Symptom	First checks
A style or template change is missing	Confirm journal_app source vs public_html output, relevant inline CSS or Vite manifest, compiled Blade views and Hostinger/CDN/browser cache.
500 error / blank application page	Inspect journal_app/storage/logs/, private .env availability, writable storage/framework and bootstrap/cache, PHP version/extensions and dependency files.
Uploaded image missing	Resolve the record path and public disk root. Check public_html/storage mapping, file existence and permissions; do not move private uploads into the web root.
PDF downloads but reader stays blank	Check PDF.js asset/worker delivery, JavaScript MIME type in .htaccess, complete matching build output, browser canvas and page/zoom controls.
Emails are not arriving	Inspect both cron jobs, mail-paused state, worker lock, SMTP configuration and failed jobs. Preserve held messages; a successful SMTP login alone is not an inbox-delivery test.
Approved user cannot access a portal	Check active account state, verified email, approved role and record-level permissions. Do not bypass verification as an audit shortcut.
Database busy / queue reservation errors	Check duplicate workers, long-running requests, locks and storage health. Review current SQLite connection settings before changing them; never replace the database with a blank file.

Post-change inspection

Check the homepage, article list, archive, editorial board, a guidance page, login boundaries and a published article. Confirm both the PDF file and the embedded reader. The /up endpoint checks framework boot; it is not a substitute for content, database and workflow checks.

10 / HOSTINGER HANDOVER

Handover checklist and evidence

• Authorized maintainers can access hPanel, File Manager, Cron Jobs and the application administration area.

• The active roots are recorded: journal_app for the private application; public_html for browser-served files.

• The exact loaded DB_DATABASE path and current cache/session/filesystem drivers are checked privately before maintenance.

• The public storage entry/mapping, private uploads and attachment path conventions are understood.

• Both minutely cron commands and the restored regular mail worker are confirmed after a release.

• SMTP secrets, application keys and account passwords are held in the approved secure handover channel.

• Database, uploads, configuration and source rollback copies have a named retention owner and a tested recovery process.

• Published article/guidance changes are made through their intended administration and workflow controls.

• No credentials, private manuscripts, signed account links or database row contents are included in this document.

Basis of this guide

Evidence	What it supports
Hostinger inspection and deployment records, 19 October 2026	Account/domain root, private and public folders, two regular cron commands, successful incremental updates and backup preservation.
Current application source and lockfiles, reviewed 24 October 2026	Folder responsibilities, exact locked package versions, storage path conventions, workflow/security rules and worker/schedule behavior.
README.md; docs/TECH_STACK.md; docs/ARCHITECTURE.md	System overview and documented environment differences. Current source takes precedence over stale operations details.
docs/MAIL_SETUP.md; config/filesystems.php; routes/console.php; scripts/hostinger-mail-worker.php	Mail transport conventions, public/private storage configuration and current scheduled/worker behavior.
Saved 19 October release reports	Database snapshot validation, preserved records/uploads/configuration and restored regular worker. These are historical checks, not a fresh uptime audit.

VERIFICATION BOUNDARY

Prepared 24 October 2026. Hosting paths and runtime wiring were last verified on 19 October. A fresh Hostinger read currently requires sign-in. Exact live SQLite filename, current installed package patch versions, present backup retention and today's cron/service status must be confirmed by an authorized maintainer. No hosting settings or journal data were changed while preparing this guide.